

INF/UFRGS – Computação Paralela | 2026-05-27

Paralelização do Algoritmo Q-Learning com OpenMP

Rayan Raddatz de Matos, Eduardo Magnus Lazuta, Enzo Lisbôa Peixoto



Agenda

- Problema e Algoritmo
- Paralelização
- Metodologia Experimental
- Resultados
- Conclusão

Problema e Algoritmo

Q-Learning: pathfinding em grid

Agente aprende a navegar de $(0, 0)$ até $(L - 1, C - 1)$ em um grid $L \times C$ com obstáculos. Ações: cima, baixo, esquerda, direita.

Q-Learning: pathfinding em grid

Agente aprende a navegar de $(0, 0)$ até $(L - 1, C - 1)$ em um grid $L \times C$ com obstáculos. Ações: cima, baixo, esquerda, direita.

A cada passo, a Q-Table é atualizada pela equação de Bellman:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Q-Learning: pathfinding em grid

Agente aprende a navegar de $(0, 0)$ até $(L - 1, C - 1)$ em um grid $L \times C$ com obstáculos. Ações: cima, baixo, esquerda, direita.

A cada passo, a Q-Table é atualizada pela equação de Bellman:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Após o treinamento, a política ótima é extraída como $\pi(s) = \arg \max_a Q(s, a)$.

Estrutura de um episódio

- Parte do estado inicial s_0
- Seleciona ação via epsilon-greedy
- Transita para s' , recebe recompensa r
- Atualiza $Q(s, a)$ pela equação de Bellman
- Repete até atingir o objetivo ou esgotar passos

Recompensas configuradas:

- $R_{\text{objetivo}} = +100$
- $R_{\text{obstaculo}} = -100$
- $R_{\text{passo}} = -1$
- $R_{\text{loop}} = -10$

Grids testados: **10x10, 50x50, 100x100**

Paralelização

Qual nível de paralelismo?

Três opções foram consideradas:

- **Ações** (argmax de 4 ações): descartada -- overhead supera o ganho

Qual nível de paralelismo?

Três opções foram consideradas:

- **Ações** (argmax de 4 ações): descartada -- overhead supera o ganho
- **Steps** (passos do episódio): descartada -- cada step depende do anterior

Qual nível de paralelismo?

Três opções foram consideradas:

- **Ações** (argmax de 4 ações): descartada -- overhead supera o ganho
- **Steps** (passos do episódio): descartada -- cada step depende do anterior
- **Episódios: escolhida** -- episódios são independentes; distribuição direta via omp for

Qual nível de paralelismo?

Três opções foram consideradas:

- **Ações** (argmax de 4 ações): descartada -- overhead supera o ganho
- **Steps** (passos do episódio): descartada -- cada step depende do anterior
- **Episódios: escolhida** -- episódios são independentes; distribuição direta via omp for

Mas paralelizar sem sincronização não basta: com T threads e E episódios, cada thread aprende como se tivesse apenas E/T episódios.

Sincronização Periódica

Cada thread mantém uma **Q-Table local**, treina um lote de `sync_interval` episódios e funde os deltas na Q-Table global:

```
#pragma omp for schedule(runtime)
for (episode = ep_start; episode < ep_end; episode++)
    run_episode(&local_ql, episode);

#pragma omp for collapse(2)
for (state ...) for (action ...) {
    delta_sum = sum_t(local[t][s][a] - snapshot);
    ql->q_table[state][action] = snapshot + delta_sum / active;
}
```

Merge por delta: divide pelas threads **ativas** ($\text{active} = \min(\text{chunk}, T)$), preservando o α efetivo quando $\text{chunk_size} < T$.

Heurísticas de `sync_interval`

Duas estratégias automáticas implementadas em `train()`:

- **sqrt**: `sync_interval = sqrt(N/T)` -- equilibra overhead de barreira e divergência local (inspirado em limites de regret para SGD paralelo)

Heurísticas de `sync_interval`

Duas estratégias automáticas implementadas em `train()`:

- **sqrt**: `sync_interval = sqrt(N/T)` -- equilibra overhead de barreira e divergência local (inspirado em limites de regret para SGD paralelo)
- **statespace**: `sync_interval = L * C` -- cada thread explora todos os estados antes de fundir

Heurísticas de `sync_interval`

Duas estratégias automáticas implementadas em `train()`:

- **sqrt**: `sync_interval = sqrt(N/T)` -- equilibra overhead de barreira e divergência local (inspirado em limites de regret para SGD paralelo)
- **statespace**: `sync_interval = L * C` -- cada thread explora todos os estados antes de fundir

Além disso, testamos valores fixos: **every10** (`i=10`) e **every1000** (`i=1000`).

HOGWILD!

Todas as threads leem e escrevem na **mesma Q-Table sem nenhuma sincronização**:

```
QLearning local_ql = *ql;  
local_ql.rand_seed = seed + tid;  
/* q_table aponta para a tabela COMPARTILHADA */  
  
#pragma omp for schedule(runtime)  
for (episode = 0; episode < N; episode++)  
    run_episode(&local_ql, episode); /* sem sincronizacao */
```

HOGWILD!

Todas as threads leem e escrevem na **mesma Q-Table sem nenhuma sincronização**:

```
QLearning local_ql = *ql;  
local_ql.rand_seed = seed + tid;  
/* q_table aponta para a tabela COMPARTILHADA */  
  
#pragma omp for schedule(runtime)  
for (episode = 0; episode < N; episode++)  
    run_episode(&local_ql, episode); /* sem sincronizacao */
```

Race conditions são "intencionais" e "inofensivas" para casos esparsos.

Metodologia Experimental

Fatores do experimento

Fator	Níveis	Motivação
Threads	1, 2, 4, 8, 16, 20, 40	Do serial ao hyperthreading completo
Tamanho grid	10, 50, 100	Espaço de estados pequeno/médio/grande
Modo de sync	sqrt, statespace, hogwild, 10, 1000	5 estratégias de paralelização
Episódios	5000, 10000	5k instável, 10k converge melhor
Obstáculos	0%, 50%	Grid livre vs. alta densidade
Schedule OMP	static, guided, dynamic	Políticas de distribuição
Chunk	0 (auto), 1, 10	Granularidade do chunking

Baseline: binário sequencial puro (sem runtime OpenMP) para $T = 1$.

Afinidade de threads e medição de tempo

Afinidade:

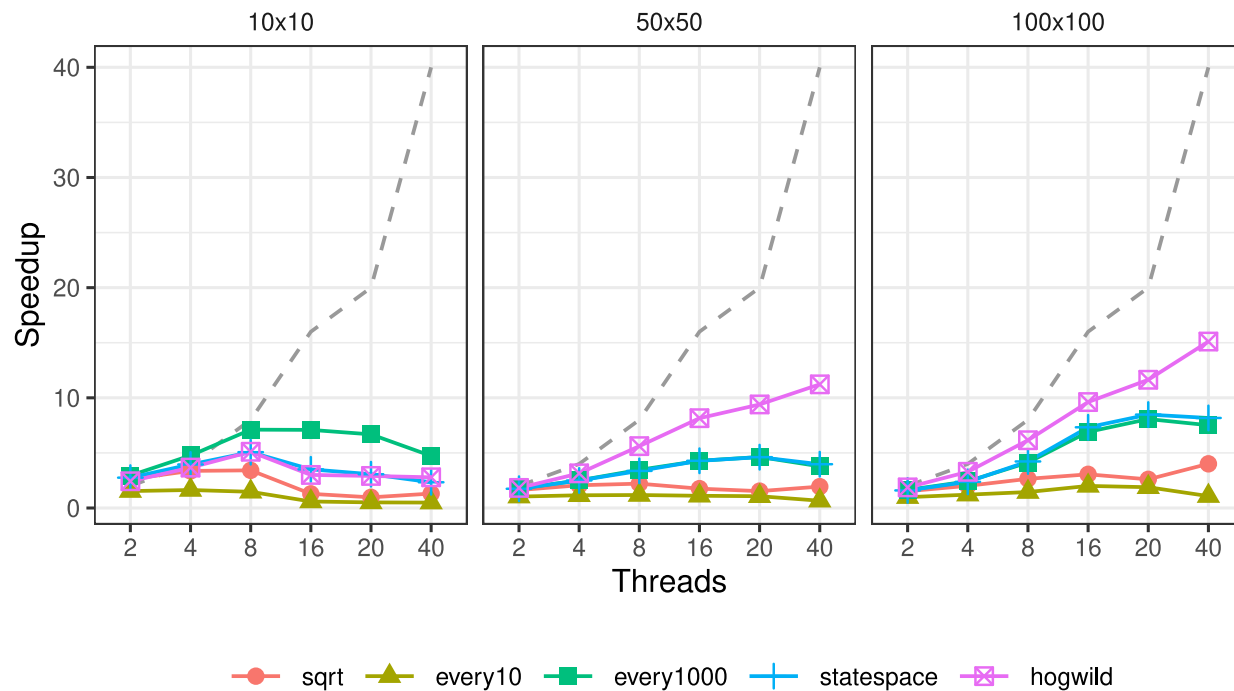
- $T \leq 20$: `OMP_PLACES=cores, OMP_PROC_BIND=close` -- 1 thread por núcleo físico
- $T = 40$: `OMP_PLACES=threads` -- hyperthreading completo

Medição de tempo:

- Apenas o tempo de `train()` -- exclui alocação, init e I/O
- Paralelo: `omp_get_wtime()` | Sequencial: `clock_gettime(CLOCK_MONOTONIC)`
- Múltiplos blocos de replicação para estimar variância do escalonador

Resultados

Speedup por modo de sincronização



Speedup: observações

- Quase ideal para 8 threads ou menos em alguns casos (hogwild, every1000).

Speedup: observações

- Quase ideal para 8 threads ou menos em alguns casos (hogwild, every1000).
- **10x10**: satura em 4--5x a partir de 8 threads; trabalho por thread insuficiente para amortizar overhead

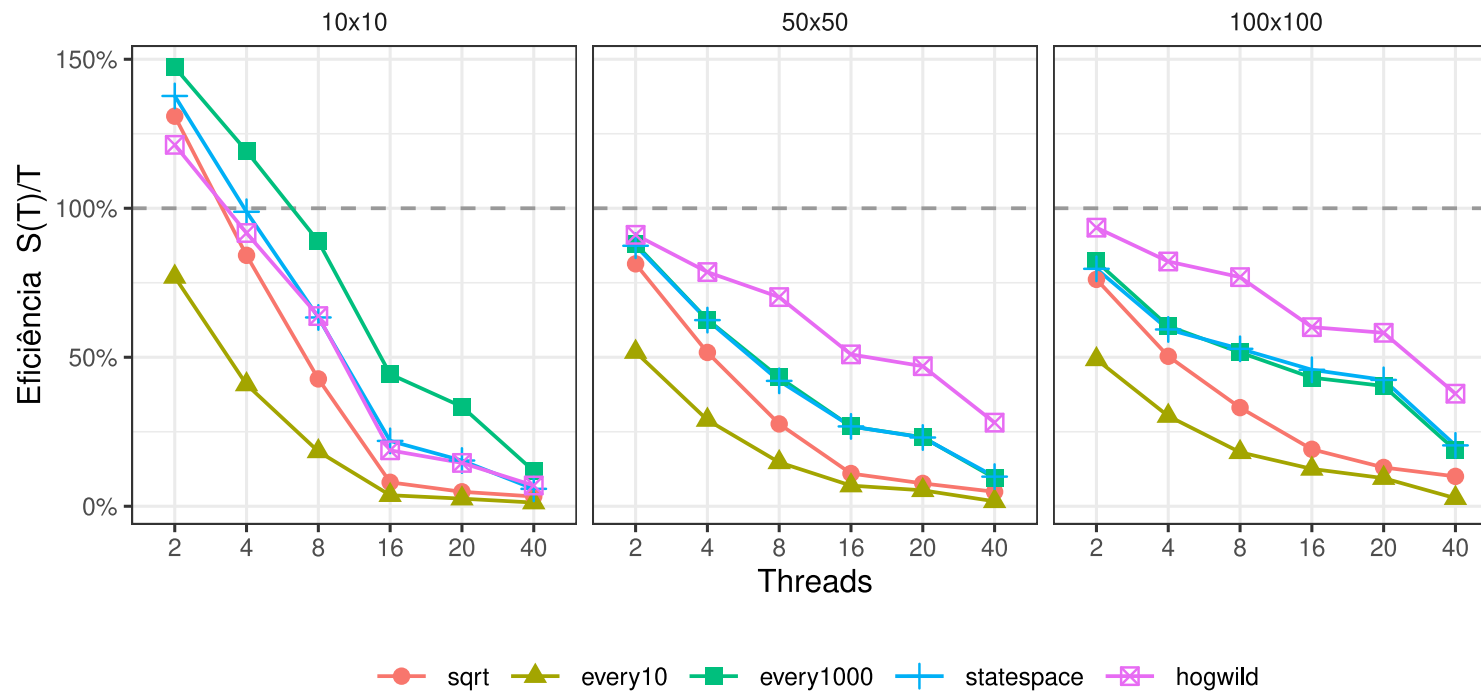
Speedup: observações

- Quase ideal para 8 threads ou menos em alguns casos (hogwild, every1000).
- **10x10**: satura em 4--5x a partir de 8 threads; trabalho por thread insuficiente para amortizar overhead
- **100x100**: HOGWILD! atinge até **17x com 40 threads** modos sincronizados ficam abaixo de 10x

Speedup: observações

- Quase ideal para 8 threads ou menos em alguns casos (hogwild, every1000).
- **10x10**: satura em 4--5x a partir de 8 threads; trabalho por thread insuficiente para amortizar overhead
- **100x100**: HOGWILD! atinge até **17x com 40 threads** modos sincronizados ficam abaixo de 10x
- Barreiras periódicas serializam parcialmente a execução nos modos sincronizados

Eficiência paralela $S(T)/T$



Eficiência: observações

- HOGWILD! mantém a **melhor eficiência** em todos os grids: 35--45% com 40 threads no 100x100

Eficiência: observações

- HOGWILD! mantém a **melhor eficiência** em todos os grids: 35--45% com 40 threads no 100x100
- **every10** (sync a cada 10 eps): **pior eficiência** pois overhead de barreiras frequentes supera o ganho

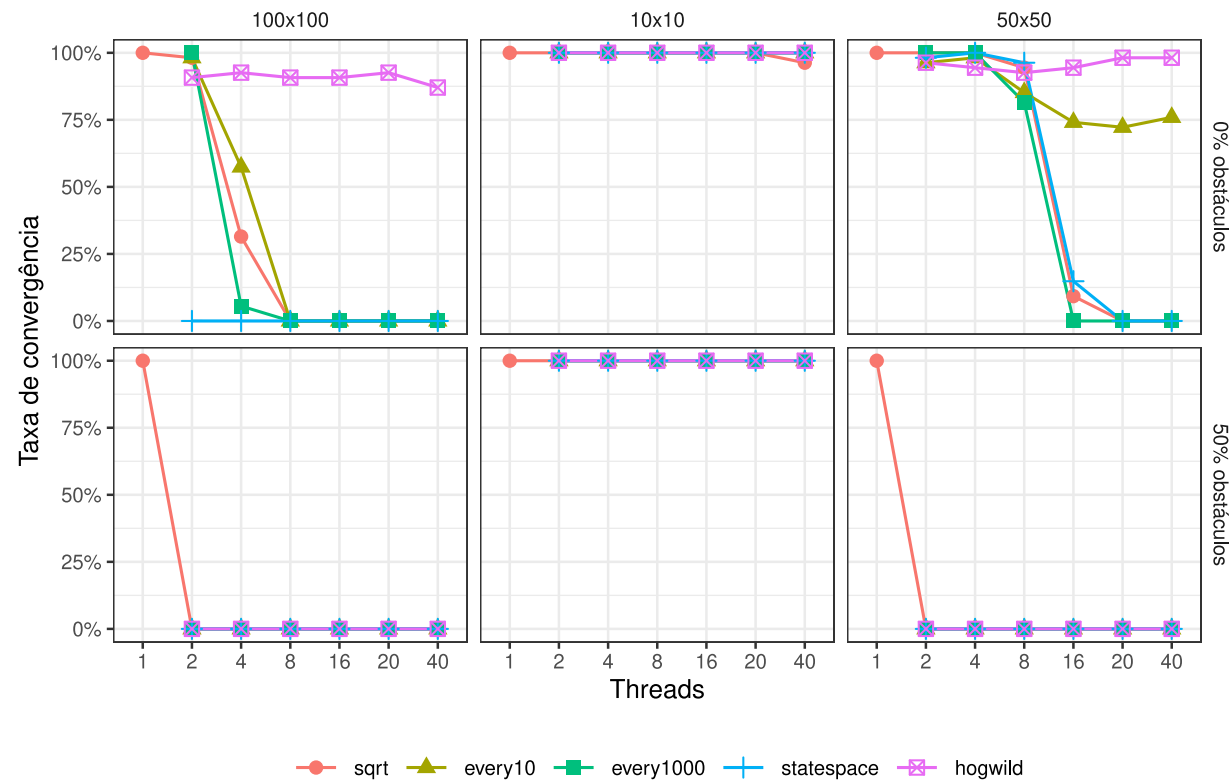
Eficiência: observações

- HOGWILD! mantém a **melhor eficiência** em todos os grids: 35--45% com 40 threads no 100x100
- **every10** (sync a cada 10 eps): **pior eficiência** pois overhead de barreiras frequentes supera o ganho
- every1000 e statespace ficam no intermediário: sincronizam menos e amortizam o custo

Eficiência: observações

- HOGWILD! mantém a **melhor eficiência** em todos os grids: 35--45% com 40 threads no 100x100
- **every10** (sync a cada 10 eps): **pior eficiência** pois overhead de barreiras frequentes supera o ganho
- every1000 e statespace ficam no intermediário: sincronizam menos e amortizam o custo
- Nenhuma configuração sustenta 50% de eficiência além de 8 threads no grid 10x10

Convergência



Convergência: causa da degradação

Com T threads e E episódios fixos, cada thread treina com E/T episódios. Quando $E/T < E_{\min}$ o agente não aprende.

Convergência: causa da degradação

Com T threads e E episódios fixos, cada thread treina com E/T episódios. Quando $E/T < E_{\min}$ o agente não aprende.

- **0% de obstáculos:** todos os modos convergem para poucos threads; degrada com o aumento
- **50% de obstáculos:** nenhuma configuração paralela convergiu para grids grandes

Convergência: causa da degradação

Com T threads e E episódios fixos, cada thread treina com E/T episódios. Quando $E/T < E_{\min}$ o agente não aprende.

- **0% de obstáculos:** todos os modos convergem para poucos threads; degrada com o aumento
- **50% de obstáculos:** nenhuma configuração paralela convergiu para grids grandes

Mesmo HOGWILD! (que acumula updates de **todas** as threads) falha com 50% de obstáculos, provavelmente pela perda de "esparsidade".

Causa da falha com 50% de obstáculos (10.000 episódios)

Causa raiz: desconto gamma anula o sinal de recompensa

Com 5000 obstáculos, o caminho mínimo tem 300 passos. Com $\gamma = 0.9$:

$$\gamma^{300} = 0.9^{300} \approx 0$$

A recompensa do goal **não chega ao estado inicial**. Mesmo em episódios bem-sucedidos, Q-target em $s_0 \approx -10$ (ainda negativo) — aumentar episódios não resolve.

Causa da falha com 50% de obstáculos (10.000 episódios)

Causa raiz: desconto gamma anula o sinal de recompensa

Com 5000 obstáculos, o caminho mínimo tem 300 passos. Com $\gamma = 0.9$:

$$\gamma^{300} = 0.9^{300} \approx 0$$

A recompensa do goal **não chega ao estado inicial**. Mesmo em episódios bem-sucedidos, Q-target em $s_0 \approx -10$ (ainda negativo) — aumentar episódios não resolve.

Medido empiricamente com 10.000 episódios e T=2:

- 570 episódios chegaram ao goal — $Q(s_0)$ ficou preso em -83 a -87 durante todas as 143 sincronizações

- Episódios falhos (9430) dominam: cada um empurra $Q(s_0)$ para -90 ; cada bem-sucedido empurra para -10

Causa da falha com 50% de obstáculos (10.000 episódios)

Causa raiz: desconto gamma anula o sinal de recompensa

Com 5000 obstáculos, o caminho mínimo tem 300 passos. Com $\gamma = 0.9$:

$$\gamma^{300} = 0.9^{300} \approx 0$$

A recompensa do goal **não chega ao estado inicial**. Mesmo em episódios bem-sucedidos, Q-target em $s_0 \approx -10$ (ainda negativo) — aumentar episódios não resolve.

Medido empiricamente com 10.000 episódios e T=2:

- 570 episódios chegaram ao goal — $Q(s_0)$ ficou preso em -83 a -87 durante todas as 143 sincronizações

- Episódios falhos (9430) dominam: cada um empurra $Q(s_0)$ para -90 ; cada bem-sucedido empurra para -10

Por que o paralelo falha universalmente e o sequencial às vezes não:

- Sequencial com certas seeds tem sequência de exploração "sortuda" que aprende uma rota local próxima ao goal
- O paralelo divide episódios entre threads e funde Q-tables, quebrando essa sequência e tornando a falha consistente

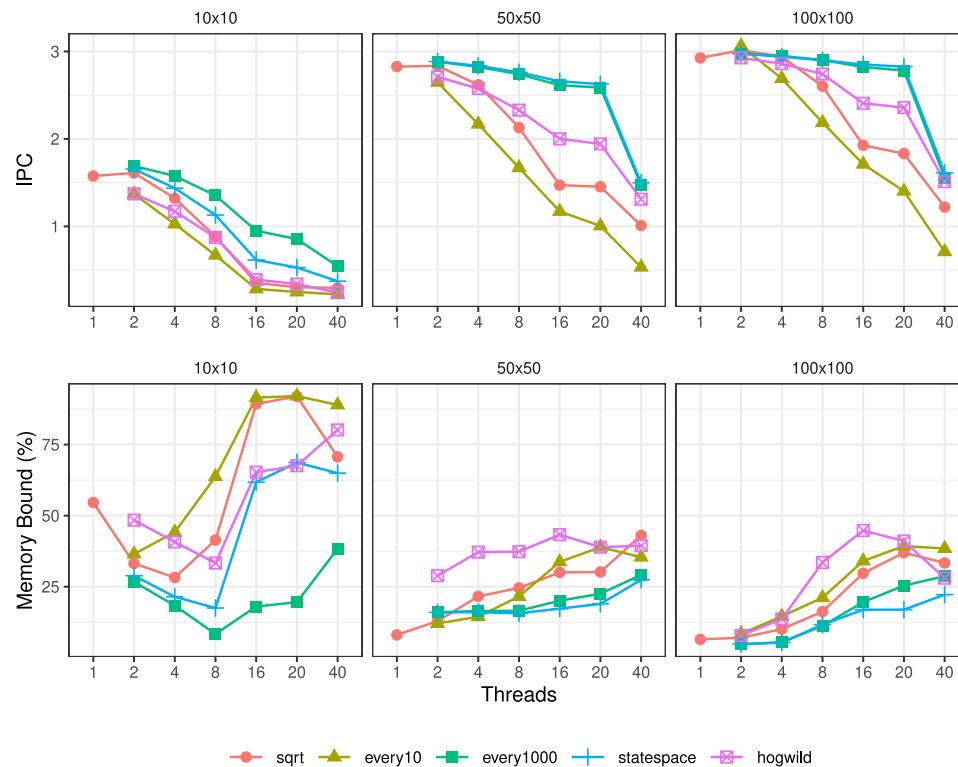
Speedup: execuções convergidas

Grid	Config	2	4	8	16	20	40
10x10	every10	1,54	1,63	1,47	0,59	0,51	0,49
10x10	every1000	2,95	4,77	7,11	7,09	6,69	4,73
10x10	hogwild	2,43	3,67	5,11	3,00	2,91	2,78
10x10	sqrt	2,62	3,37	3,42	1,29	0,97	1,32
10x10	statespace	2,75	3,95	5,07	3,51	3,07	2,34

Grid	Config	2	4	8	16	20	40
50x50	every10	1,08	1,11	0,98	0,65	0,59	0,38
50x50	every1000	2,21	2,96	3,76	—	—	—
50x50	hogwild	2,32	4,04	7,40	9,19	10,12	10,65
50x50	sqrt	1,98	2,33	2,20	1,19	—	—
50x50	statespace	2,20	2,91	3,58	4,16	—	—
100x100	every10	0,90	0,85	—	—	—	—
100x100	every1000	1,97	2,49	—	—	—	—
100x100	hogwild	2,43	4,24	8,00	11,45	13,88	18,14
100x100	sqrt	1,77	1,99	—	—	—	—

Speedup medio (10k ep., 0% obstaculos). “—” = nenhuma execucao convergiu.

VTune: IPC e Memory Bound



Análise de Cache (10x10)

Q-Table do 10x10 ocupa **3,2 KB** (100 estados x 4 ações x 8 bytes) -- cabe inteira em L1. Taxa de LLC-miss medida com perf stat:

T	sq-L1	sq-LLC	spc-L1	spc-LLC	hog-L1	hog-LLC
2	1,12%	0,80%	1,10%	1,11%	7,42%	0,44%
4	3,87%	0,70%	4,10%	0,79%	8,53%	0,18%
8	6,37%	0,17%	5,15%	0,18%	23,20%	0,11%
16	3,95%	30,9%	4,64%	23,1%	5,99%	36,7%
20	5,08%	29,1%	11,09%	22,9%	1,38%	36,4%
40	1,99%	40,7%	1,98%	28,5%	4,17%	35,9%

Piora quando o número de threads ultrapassa 8, pois o gargalo se torna **tráfego de cache**, não falta de trabalho.

Análise de Cache: interpretação

Para $T \leq 8$: Todos modos tem LLC-miss **menor** pois Q-Table única de 3,2 KB fica estável em L1/L2. Modos sincronizados tem um miss L1 um pouco maior pois tem várias cópias das tableas.

Análise de Cache: interpretação

Para $T \leq 8$: Todos modos tem LLC-miss **menor** pois Q-Table única de 3,2 KB fica estável em L1/L2. Modos sincronizados tem um miss L1 um pouco maior pois tem várias cópias das tableas.

Sockets: Hype é dual-socket (2x Xeon E5-2650 v3, 10 núcleos/socket). Com `OMP_PROC_BIND=close`, $T \leq 10$ fica inteiro no socket 0; $T = 16$ já usa 6 núcleos do socket 1. Por isso vemos mais LLC-miss.

Análise de Cache: interpretação

Para $T \leq 8$: Todos modos tem LLC-miss **menor** pois Q-Table única de 3,2 KB fica estável em L1/L2. Modos sincronizados tem um miss L1 um pouco maior pois tem várias cópias das tableas.

Sockets: Hype é dual-socket (2x Xeon E5-2650 v3, 10 núcleos/socket). Com `OMP_PROC_BIND=close`, $T \leq 10$ fica inteiro no socket 0; $T = 16$ já usa 6 núcleos do socket 1. Por isso vemos mais LLC-miss.

Para $T \geq 16$: hogwild é o pior (36,7%), sem cópia local, acessa frequentemente o socket 1. Statespace fica em 23,1% porque os threads do socket 1 carregam a cópia local uma vez e a reutilizam por 100 episódios.

Análise de Cache (100x100)

Q-Table ocupa **320 KB**, excede L1 (32 KB) e L2 (256 KB), cabe no LLC (20 MB).

Baseline sequencial: 2,82% de L1-miss (vs. 0,18% no 10x10).

L1-miss

T	sqrt	ev10	ev1k	spc	hog
2	5,63%	10,27%	3,39%	3,28%	4,50%
4	6,27%	10,77%	2,91%	2,62%	4,30%
8	7,80%	11,59%	2,64%	2,09%	5,41%
16	11,79%	13,74%	1,98%	2,18%	4,36%
20	12,85%	14,59%	2,83%	2,17%	3,78%
40	11,03%	12,43%	3,01%	2,01%	2,56%

LLC-miss

T	sqrt	ev10	ev1k	spc	hog
2	0,06%	0,01%	0,08%	0,08%	0,00%
4	0,12%	0,03%	0,29%	0,05%	0,00%
8	0,01%	0,02%	0,04%	0,05%	0,20%
16	7,01%	6,81%	3,12%	1,59%	35,44%
20	22,43%	23,00%	14,03%	5,43%	33,49%
40	27,65%	28,38%	16,47%	2,61%	60,77%

Media dos tres schedules.

Análise de Cache 100x100: interpretação

- $T \leq 8$: todos no socket 0, LLC-miss $< 0,3\%$ em todos os modos

Análise de Cache 100x100: interpretação

- $T \leq 8$: todos no socket 0, LLC-miss $< 0,3\%$ em todos os modos
- **Hogwild** (60,8% com 40T): sem cópia local, cada escrita do socket 1 invalida linhas no socket 0

Análise de Cache 100x100: interpretação

- $T \leq 8$: todos no socket 0, LLC-miss $< 0,3\%$ em todos os modos
- **Hogwild** (60,8% com 40T): sem cópia local, cada escrita do socket 1 invalida linhas no socket 0
- **Statespace** (2,6% com 40T): intervalo = 10^4 , episódios = uma sync total; threads do socket 1 carregam cópia local uma vez e não voltam

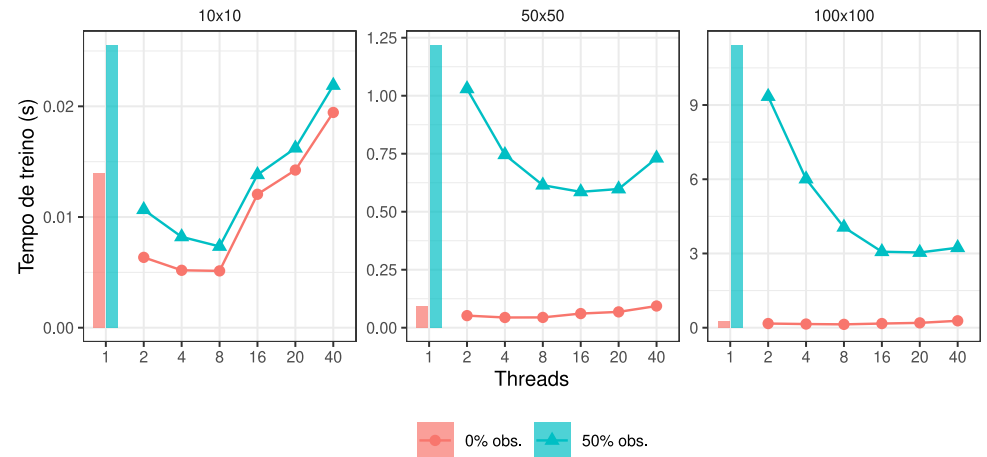
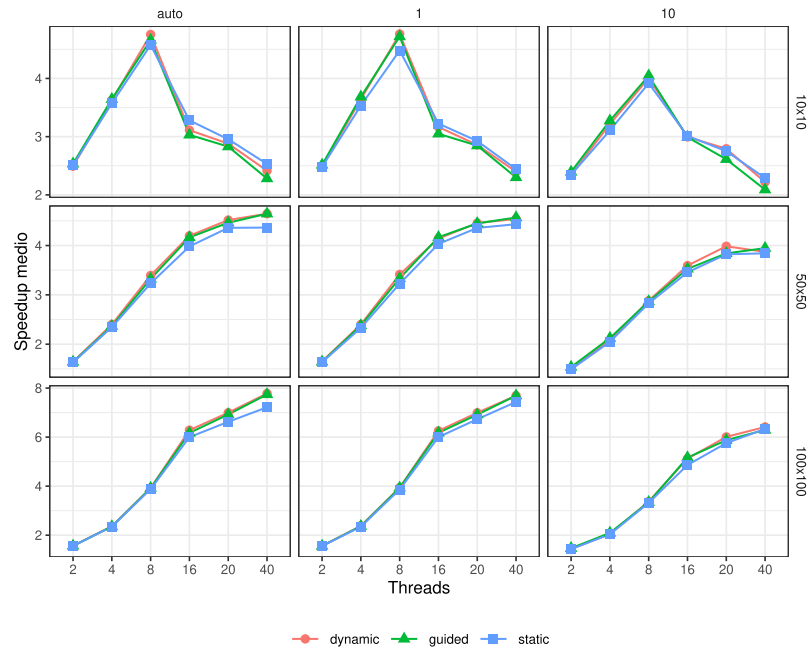
Análise de Cache 100x100: interpretação

- $T \leq 8$: todos no socket 0, LLC-miss $< 0,3\%$ em todos os modos
- **Hogwild** (60,8% com 40T): sem cópia local, cada escrita do socket 1 invalida linhas no socket 0
- **Statespace** (2,6% com 40T): intervalo = 10^4 , episódios = uma sync total; threads do socket 1 carregam cópia local uma vez e não voltam
- **Every1000** (16,5%) melhor que sqrt (27,7%) -- menos cruzamentos de QPI por merge

Análise de Cache 100x100: interpretação

- $T \leq 8$: todos no socket 0, LLC-miss $< 0,3\%$ em todos os modos
- **Hogwild** (60,8% com 40T): sem cópia local, cada escrita do socket 1 invalida linhas no socket 0
- **Statespace** (2,6% com 40T): intervalo = 10^4 , episódios = uma sync total; threads do socket 1 carregam cópia local uma vez e não voltam
- **Every1000** (16,5%) melhor que sqrt (27,7%) -- menos cruzamentos de QPI por merge
- L1-miss alto no **every10** (10-15%): copia 320 KB a cada 10 episódios, expulsa dados úteis do L1 repetidamente

Schedule, chunk e obstáculos



Schedule e chunk: não são fatores dominantes

- Os três schedules produzem speedup **praticamente idêntico**: durações de episódio homogêneas, carga equilibrada independente da política
- Diferença geral permanece **<2% no speedup médio**
- Chunk=1: leve degradação em grids pequenos (overhead por episódio); chunk=10 e auto se comportam de forma equivalente
- Obstáculos: comparar speedup entre 0% e 50% é **injusto**, ganho aparente com 50% reflete apenas que há mais trabalho total a dividir

Conclusão

Conclusão

- **Gargalos principal:** divisão de episódios e miss. Com T threads e E fixos, cada thread treina com E/T -- abaixo do limiar, agente não converge
- **Problemas de memória entre cache:** gargalo é coerência de cache, LLC-miss explode de <1,2% para 23-40% após $T = 8$
- **HOGWILD!** obteve maior speedup bruto (até **17x** com 40 threads em 100x100), mas não converge com 50% de obstáculos
- Modos sincronizados com intervalos longos (**every1000, statespace**) preservam melhor o comportamento de cache entre syncs, mas convergem menos
- Schedule e chunk: **não são fatores dominantes** (<2% de impacto no speedup)

Perguntas?